

# TripGenie - Documentation Technique Détaillée (Version Finale)

**Auteur :** Alexis Loties (Projet Solo - Certification)

## 0. User Personas

Avant de concevoir l'architecture technique, il est essentiel d'identifier précisément qui va utiliser TripGenie et dans quel contexte. Ces personas ont été construits à partir du problème identifié en Phase 1 : la fragmentation de la planification de voyage et les 2 à 4 heures perdues pour organiser un simple week-end.

### PERSONA 1 — LE VOYAGEUR SOLO SPONTANÉ

| Attribut               | Détail                                                                                                                             |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Nom fictif             | Lucas, 28 ans — développeur freelance                                                                                              |
| Comportement           | Voyage 4 à 6 fois par an, souvent décidé à J-7. Utilise Skyscanner, Booking et Google Maps en parallèle. Perd du temps à comparer. |
| Frustration principale | Trop d'onglets ouverts, résultats dispersés, difficulté à trouver des activités locales pendant son séjour.                        |
| Objectif               | Obtenir un pack complet (vol + hôtel + activités) en moins de 5 minutes, adapté à son budget et son ambiance.                      |
| Usage TripGenie        | Utilise le chatbot pour décrire son voyage, choisit parmi 3 destinations proposées, sauvegarde son itinéraire.                     |
| Device principal       | Ordinateur portable + smartphone (responsive indispensable).                                                                       |

## PERSONA 2 — LE GROUPE D'AMIS INDÉCIS

| Attribut               | Détail                                                                                                                     |
|------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Nom fictif             | Marie, 25 ans — organisatrice du groupe de 4 amis                                                                          |
| Comportement           | Organise les voyages pour son groupe. Doit valider les choix avec tout le monde via WhatsApp. Processus long et frustrant. |
| Frustration principale | Impossible de faire voter le groupe facilement. Tout le monde donne son avis sur des liens différents.                     |
| Objectif               | Générer un pack, partager un lien unique au groupe, laisser chacun voter sur les options proposées.                        |
| Usage TripGenie        | Crée le voyage, partage le lien de vote, consulte les résultats et valide le pack final.                                   |
| Device principal       | Smartphone principalement (vote mobile critique).                                                                          |

## PERSONA 3 — LE VOYAGEUR BUDGET PRÉCIS

| Attribut               | Détail                                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Nom fictif             | Théo, 22 ans — étudiant, budget serré                                                                                    |
| Comportement           | Voyage avec un budget max très précis. Passe des heures à comparer les prix. Abandonne souvent faute de solution simple. |
| Frustration principale | Les outils existants ne filtrent pas efficacement par budget total (vol + hôtel + activités).                            |
| Objectif               | Entrer un budget global et obtenir automatiquement un pack cohérent qui respecte cette contrainte.                       |
| Usage TripGenie        | Spécifie son budget au chatbot, reçoit des packs triés par score de correspondance prix/qualité.                         |
| Device principal       | Smartphone (usage mobile-first).                                                                                         |

---

# 1. User Stories & Critères d'Acceptation

---

## Méthode de Priorisation — MoSCoW

| Priorité      | Signification                                                           |
|---------------|-------------------------------------------------------------------------|
| <b>MUST</b>   | Indispensable au MVP — le produit ne fonctionne pas sans.               |
| <b>SHOULD</b> | Important mais pas bloquant — livré dans la même itération si possible. |
| <b>COULD</b>  | Souhaitable — ajouté si le temps le permet.                             |
| <b>WON'T</b>  | Hors scope MVP — prévu pour une version future.                         |

## ÉPOPÉE 1 — AUTHENTIFICATION & COMPTE UTILISATEUR

### US-01 — Inscription

- **User Story** : En tant que visiteur, je veux créer un compte avec mon email et un mot de passe afin d'accéder aux fonctionnalités personnalisées de TripGenie.
- **Priorité** : MUST
- **Critères d'acceptation** : ✓ Le formulaire valide le format email en temps réel. ✓ Le mot de passe est haché avec bcrypt avant stockage. ✓ Un message de confirmation est affiché après inscription réussie. ✓ L'email déjà utilisé déclenche un message d'erreur explicite (409). ✓ Un token JWT est généré et retourné à la connexion.
- **Endpoint** : `POST /api/auth/register`

### US-02 — Connexion

- **User Story** : En tant qu'utilisateur inscrit, je veux me connecter avec mon email et mon mot de passe afin d'accéder à mon historique et mes voyages sauvegardés.
- **Priorité** : MUST
- **Critères d'acceptation** : ✓ Connexion réussie retourne un token JWT valide 24h. ✓ Mot de passe incorrect retourne une erreur 401 sans détail sensible. ✓ Token stocké côté client (localStorage ou cookie httpOnly). ✓ Toutes les routes protégées vérifient le token via middleware.

- **Endpoint :** POST /api/auth/login

## ÉPOPÉE 2 — CHATBOT D'ONBOARDING IA

### US-03 — Collecte du profil voyage via chatbot

- **User Story :** En tant qu'utilisateur, je veux décrire mon voyage en langage naturel (groupe, budget, durée, ambiance) afin que TripGenie comprenne mon besoin sans formulaire complexe.
- **Priorité :** MUST
- **Critères d'acceptation :** ✓ Le chatbot pose 5 questions maximum (mode, budget, durée, groupe, ambiance). ✓ Le système extrait correctement les paramètres dans au moins 90% des cas. ✓ En cas d'ambiguïté, le bot demande une clarification ciblée. ✓ Un fallback générique est déclenché si l'API LLM est indisponible. ✓ L'historique de la conversation est affiché dans l'interface.
- **Endpoint :** POST /api/chat/message

### US-04 — Proposition de destinations

- **User Story :** En tant qu'utilisateur ayant complété l'onboarding, je veux voir 3 destinations proposées avec leur score de correspondance afin de choisir celle qui me convient le mieux.
- **Priorité :** MUST
- **Critères d'acceptation :** ✓ 3 destinations sont affichées avec nom, score (%) et courte description. ✓ Chaque destination est cliquable pour générer le pack complet. ✓ Le score est calculé selon les critères pondérés par mode de voyage. ✓ Le résultat s'affiche en moins de 10 secondes.
- **Endpoint :** POST /api/destinations/suggest

## ÉPOPÉE 3 — GÉNÉRATION DU PACK VOYAGE

### US-05 — Génération du pack complet

- **User Story :** En tant qu'utilisateur ayant choisi une destination, je veux recevoir automatiquement un pack complet (vol + hôtel + activités + événements locaux) afin d'avoir tout en un seul endroit.
- **Priorité :** MUST

- **Critères d'acceptation** : ✓ Le pack contient au minimum : 1 option de vol, 1 hôtel, 3 activités, événements locaux via PredictHQ. ✓ Le pack est généré en moins de 15 secondes. ✓ En cas d'indisponibilité d'une API, les autres éléments sont quand même affichés. ✓ Les prix sont affichés en euros avec une mention "estimation". ✓ Chaque élément du pack affiche sa source (API utilisée).
- **Endpoint** : `POST /api/packs/generate`

## US-06 — Scoring et classement des packs

- **User Story** : En tant qu'utilisateur, je veux voir le score de pertinence de chaque pack afin de comprendre pourquoi une option est recommandée par rapport à une autre.
- **Priorité** : MUST
- **Critères d'acceptation** : ✓ Chaque pack affiche un score global (0–100) et un détail par critère. ✓ Les critères sont : adéquation budget, qualité hôtel, richesse événements, originalité. ✓ La pondération varie selon le mode choisi (solo, groupe, famille...). ✓ Le scoring est une fonction pure testable indépendamment des APIs.
- **Endpoint** : Fonction interne : `scoringService.calculate(pack, profile)`

## ÉPOPÉE 4 — SAUVEGARDE & HISTORIQUE

### US-07 — Sauvegarde d'un voyage

- **User Story** : En tant qu'utilisateur connecté, je veux sauvegarder un pack voyage afin de le retrouver plus tard dans mon historique.
- **Priorité** : SHOULD
- **Critères d'acceptation** : ✓ Bouton "Sauvegarder" visible sur chaque pack généré. ✓ Le voyage est stocké en base (PostgreSQL) associé à l'utilisateur. ✓ Un message de confirmation s'affiche après la sauvegarde. ✓ La sauvegarde est accessible depuis la page "Mes voyages".
- **Endpoint** : `POST /api/trips` — `GET /api/trips`

## ÉPOPÉE 5 — VOTE COLLECTIF (FONCTIONNALITÉ SOCIALE)

### US-08 — Partage du voyage pour vote de groupe

- **User Story** : En tant qu'organisateur d'un voyage de groupe, je veux partager un lien unique à mes amis afin qu'ils puissent voter sur chaque élément du pack sans avoir à créer un compte.
- **Priorité** : SHOULD
- **Critères d'acceptation** : ✓ Un lien unique est généré pour chaque pack (token UUID). ✓ Les participants accèdent au pack sans inscription. ✓ Chaque participant peut voter (👍/👎) sur chaque élément. ✓ L'organisateur voit les résultats des votes en temps réel. ✓ Le lien expire après 7 jours ou après validation du pack.
- **Endpoint** : `POST /api/trips/:id/share` — `POST /api/trips/:id/vote`

## Récapitulatif MoSCoW

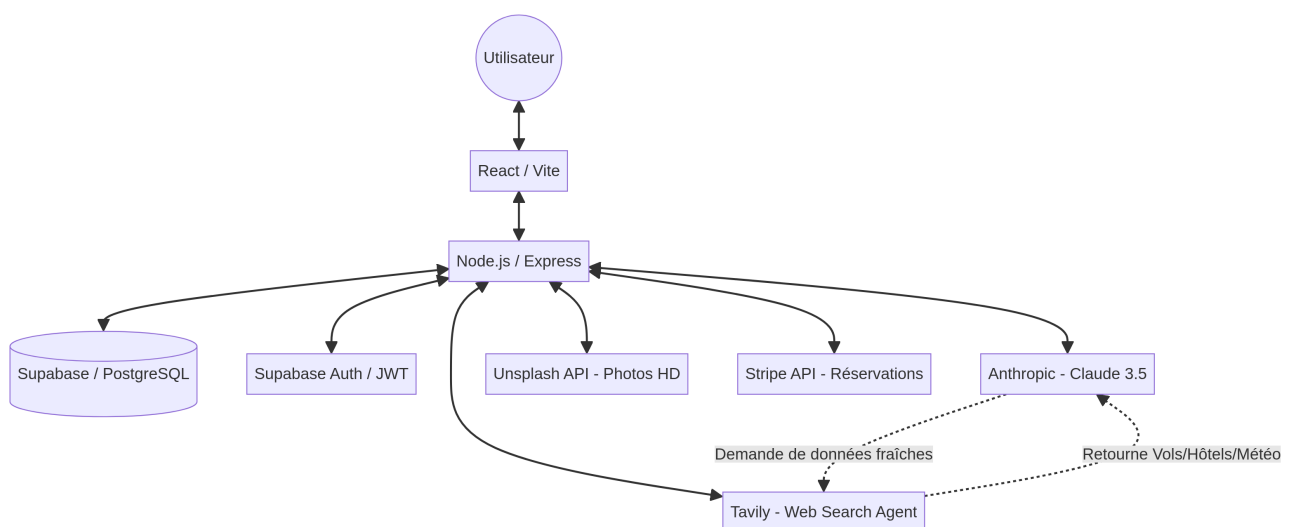
| ID    | Priorité | User Story résumée                         | Épopée      |
|-------|----------|--------------------------------------------|-------------|
| US-01 | MUST     | Inscription email/mot de passe             | Auth        |
| US-02 | MUST     | Connexion et token JWT                     | Auth        |
| US-03 | MUST     | Collecte profil voyage via chatbot         | Chatbot IA  |
| US-04 | MUST     | Proposition de 3 destinations scorées      | Chatbot IA  |
| US-05 | MUST     | Génération pack complet en < 15s           | Pack Voyage |
| US-06 | MUST     | Scoring et classement des packs            | Pack Voyage |
| US-07 | SHOULD   | Sauvegarde et historique des voyages       | Sauvegarde  |
| US-08 | SHOULD   | Vote collectif via lien de partage         | Social      |
| US-09 | COULD    | Modification manuelle d'un élément du pack | Pack Voyage |
| US-10 | WON'T    | Réservation avec paiement intégré          | Hors MVP    |

## 2. Architecture du Système

### Vue d'ensemble

TripGenie suit une architecture client-serveur découplée en trois couches principales : un frontend React (SPA), une API REST Node.js/Express, et une base de données PostgreSQL. Les APIs externes (LLM, PredictHQ) sont orchestrées exclusivement côté serveur pour sécuriser les clés d'API.

### Schéma d'Architecture



### Description du Flux de Données

- Interaction Utilisateur** : L'utilisateur envoie une requête via le Frontend.
- Orchestration Backend** : Le Backend vérifie l'identité via Supabase Auth.
- Intelligence Agentique** : Le Backend sollicite Claude 3.5 qui orchestre une recherche via Tavily pour obtenir des données réelles (météo, vols, hôtels).
- Enrichissement Visuel** : Le Backend récupère des visuels via Unsplash pour chaque destination.
- Persistence** : Le pack final est stocké dans la DB (Supabase) pour consultation ultérieure.
- Transaction (Optionnel)** : Le flux se termine par une session Stripe pour la validation de la réservation.

# Stack Technologique Finale & Justifications

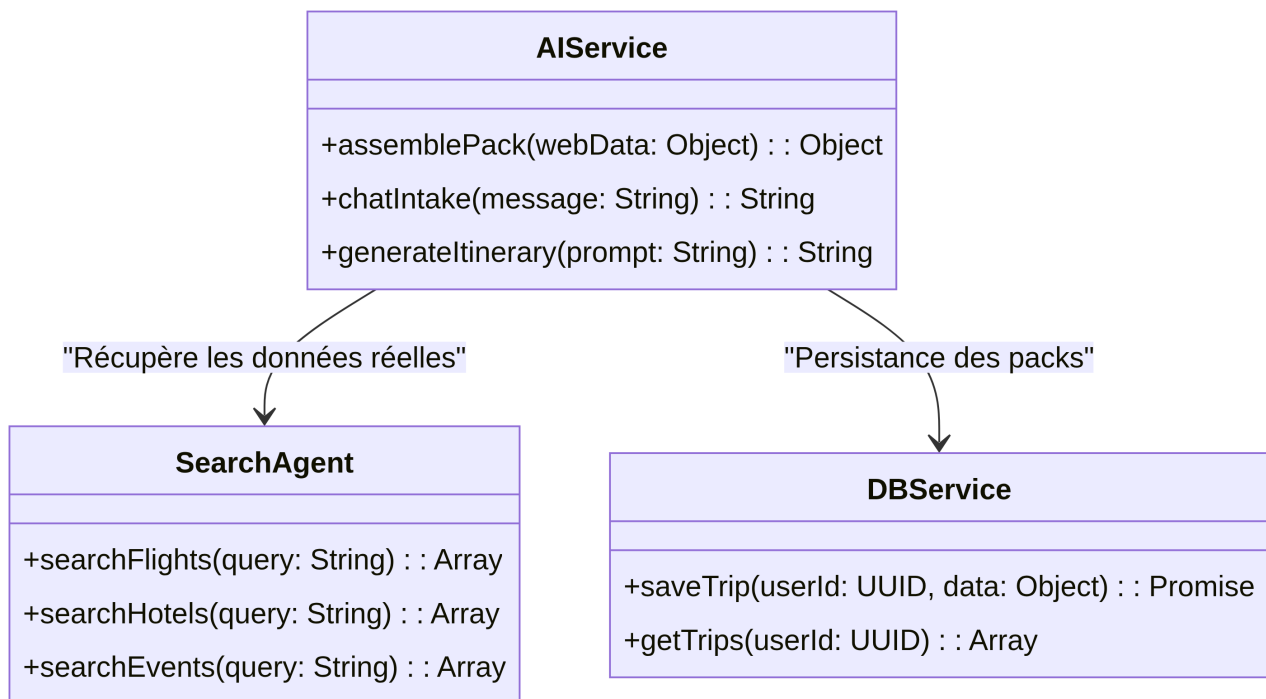
| Couche          | Technologie             | Justification technique                                                              |
|-----------------|-------------------------|--------------------------------------------------------------------------------------|
| Frontend        | React 18 + Vite         | SPA performante, rechargement instantané avec Vite HMR, écosystème riche.            |
| État global     | Zustand                 | Moins verbeux que Redux, persistance built-in pour les sessions utilisateur.         |
| Style           | Tailwind CSS            | Design system cohérent, responsive mobile-first, pas de CSS custom à maintenir.      |
| Backend         | Node.js + Express       | API REST claire, stack JS unifiée front/back, nombreux middlewares disponibles.      |
| ORM             | Prisma                  | Typage fort, migrations automatiques, requêtes lisibles et sécurisées.               |
| Base de données | PostgreSQL / Supabase   | Relationnel pour les données structurées, JSONB pour les packs complexes. RLS natif. |
| Auth            | JWT + bcrypt            | Stateless côté serveur, scalable. bcrypt pour hachage robuste des mots de passe.     |
| Validation      | Zod                     | Validation schéma TypeScript-friendly, messages d'erreur clairs pour le client.      |
| IA              | API LLM (Claude/Gemini) | Compréhension langage naturel, extraction de paramètres structurés (JSON mode).      |
| Événements      | PredictHQ API           | Données événements temps réel, sandbox disponible, documentation complète.           |
| Tests           | Jest + Supertest        | Tests unitaires des services, tests d'intégration des routes HTTP.                   |
| Déploiement     | Railway / Render        | Déploiement gratuit, support Node.js + PostgreSQL, CD automatique depuis GitHub.     |



### 3. Conception Technique (Composants & Classes)

#### Architecture des Classes (Backend - Node.js)

Le backend est structuré en services modulaires. Voici les classes clés qui pilotent la logique métier :



- **AIService** : C'est le cerveau de l'application. Sa méthode `assemblePack` fusionne les données web avec le contexte utilisateur.
- **SearchAgent** : Gère l'interface avec Tavily AI. Ses méthodes extraient les meilleurs prix et disponibilités en temps réel.
- **DBService** : Gère les interactions avec Supabase pour la lecture/écriture sécurisée des données.

#### Composants de l'Interface (Frontend - React)

L'UI est découpée en composants autonomes pour garantir une expérience fluide :

- **ChatWidget** : Interface de dialogue. Il capture les besoins de l'utilisateur et affiche les réponses de l'IA.
- **ResultsView** : Orchestre l'affichage du pack voyage. Il utilise des "sub-components" pour rendre les cartes d'hôtels et de vols.

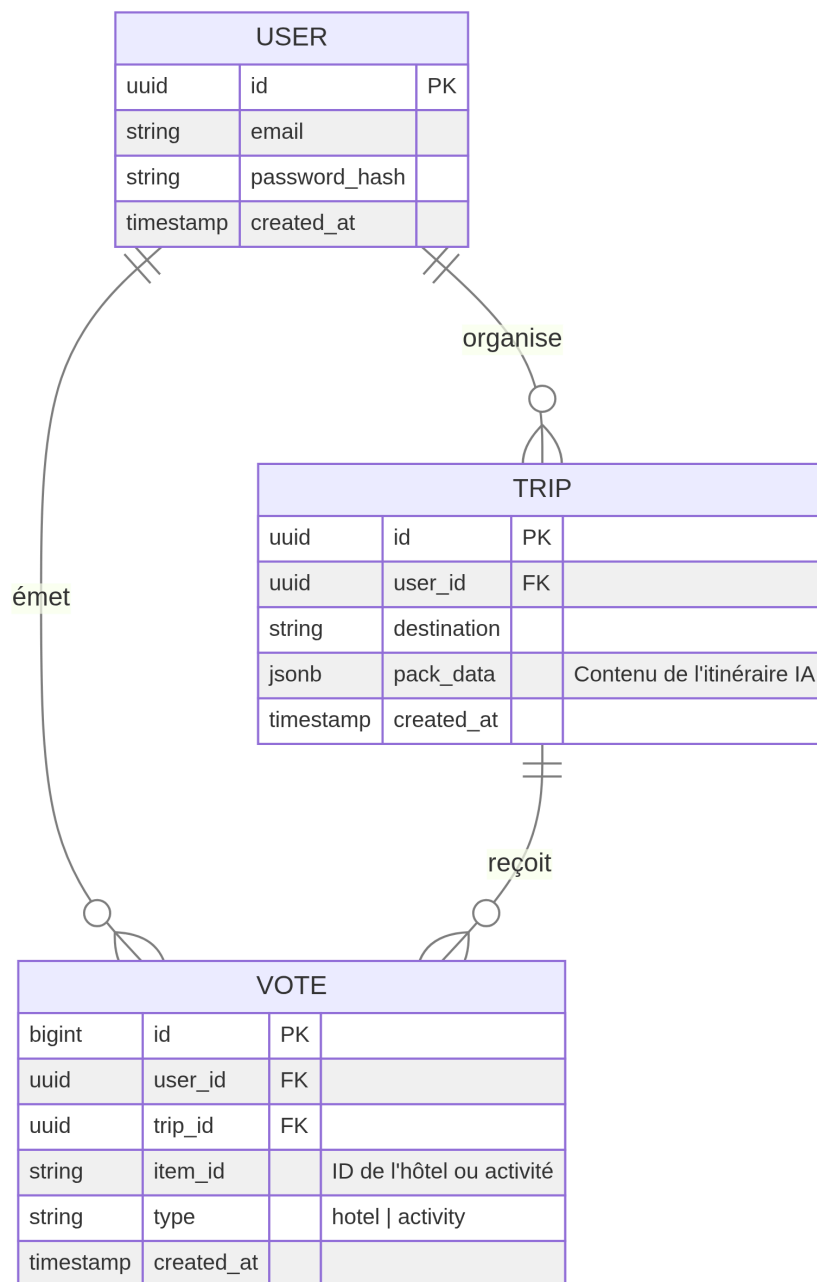
- **TripCard** : Affiche les détails HD d'une suggestion (Unsplash) et gère l'état local des votes utilisateur.
  - **AuthLayout** : Gère les formulaires de connexion/inscription via le provider Supabase.
- 

## 4. Conception de la Base de Données (Modèle Entité-Relation)

---

Le choix de **PostgreSQL** a été fait pour garantir l'intégrité des données et la flexibilité du stockage des itinéraires via le type `JSONB`.

## Diagramme Entité-Relation (ERD)



## Entités et Relations Détaillées

Le schéma PostgreSQL est conçu pour être extensible. Les packs voyage sont stockés en JSONB pour préserver la flexibilité des données APIs externes tout en bénéficiant des relations SQL pour les utilisateurs et les votes.

TABLE : users

| Colonne       | Type         | Contrainte       | Description                               |
|---------------|--------------|------------------|-------------------------------------------|
| id            | UUID         | PK               | Identifiant unique généré automatiquement |
| email         | VARCHAR(255) | UNIQUE, NOT NULL | Email de connexion — validé côté API      |
| password_hash | VARCHAR(255) | NOT NULL         | Mot de passe haché via bcrypt (12 rounds) |
| created_at    | TIMESTAMP    | DEFAULT NOW()    | Date de création du compte                |
| updated_at    | TIMESTAMP    | DEFAULT NOW()    | Dernière mise à jour automatique          |

TABLE : trips

| Colonne       | Type         | Contrainte     | Description                                                |
|---------------|--------------|----------------|------------------------------------------------------------|
| id            | UUID         | PK             | Identifiant unique du voyage                               |
| user_id       | UUID         | FK → users(id) | Propriétaire du voyage                                     |
| destination   | VARCHAR(100) | NOT NULL       | Destination choisie                                        |
| profile       | JSONB        | NOT NULL       | Profil voyage : {mode, budget, duration, group_size, vibe} |
| pack          | JSONB        | NOT NULL       | Pack complet : {flights, hotel, activities, events}        |
| score         | INTEGER      | CHECK (0-100)  | Score de correspondance calculé                            |
| share_token   | UUID         | UNIQUE         | Token de partage pour vote collectif                       |
| share_expires | TIMESTAMP    | NULLABLE       | Expiration du lien de partage (J+7)                        |
| created_at    | TIMESTAMP    | DEFAULT NOW()  | Date de génération du pack                                 |

**TABLE : votes**

| Colonne      | Type         | Contrainte        | Description                                             |
|--------------|--------------|-------------------|---------------------------------------------------------|
| id           | UUID         | PK                | Identifiant du vote                                     |
| trip_id      | UUID         | FK →<br>trips(id) | Voyage concerné                                         |
| voter_token  | UUID         | NOT NULL          | Token anonyme du votant (pas de compte requis)          |
| element_type | VARCHAR(50)  | NOT NULL          | Type d'élément voté : flight , hotel , activity , event |
| element_id   | VARCHAR(100) | NOT NULL          | Identifiant de l'élément dans le pack                   |
| value        | SMALLINT     | CHECK (0 or 1)    | 1 = vote positif, 0 = vote négatif                      |
| created_at   | TIMESTAMP    | DEFAULT<br>NOW()  | Horodatage du vote                                      |

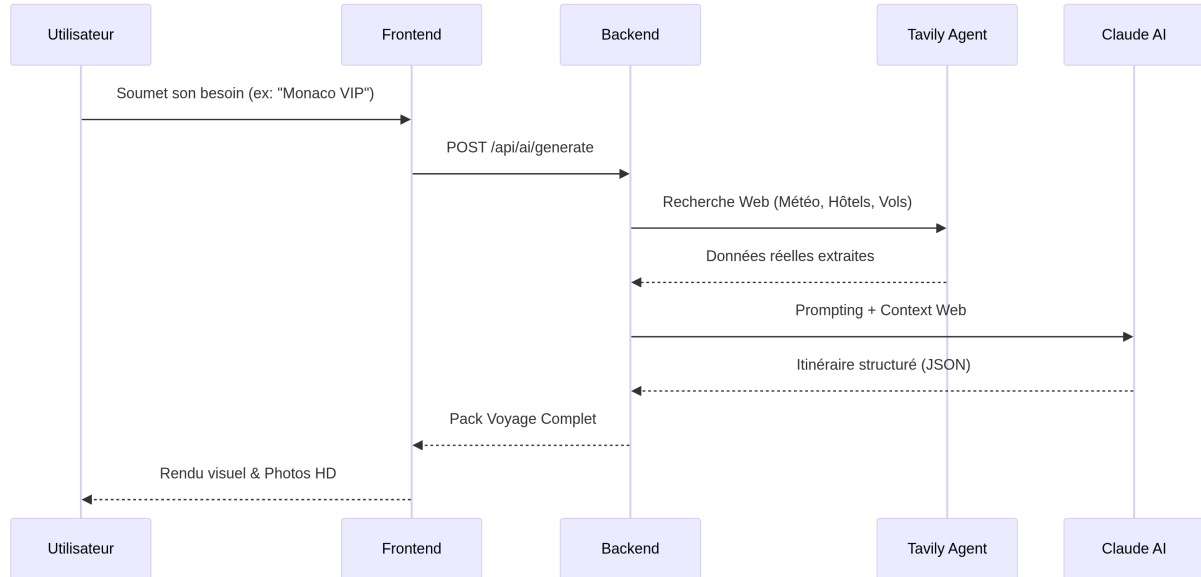
**TABLE : chat\_sessions**

| Colonne    | Type        | Contrainte          | Description                                                 |
|------------|-------------|---------------------|-------------------------------------------------------------|
| id         | UUID        | PK                  | Identifiant de la session chatbot                           |
| user_id    | UUID        | FK →<br>users(id)   | Utilisateur associé                                         |
| messages   | JSONB       | NOT NULL            | Historique complet : [{role, content, timestamp}]           |
| extracted  | JSONB       | NULLABLE            | Paramètres extraits : {mode, budget, duration, group, vibe} |
| status     | VARCHAR(20) | DEFAULT<br>'active' | Statut : active , completed , abandoned                     |
| created_at | TIMESTAMP   | DEFAULT<br>NOW()    | Début de la session                                         |

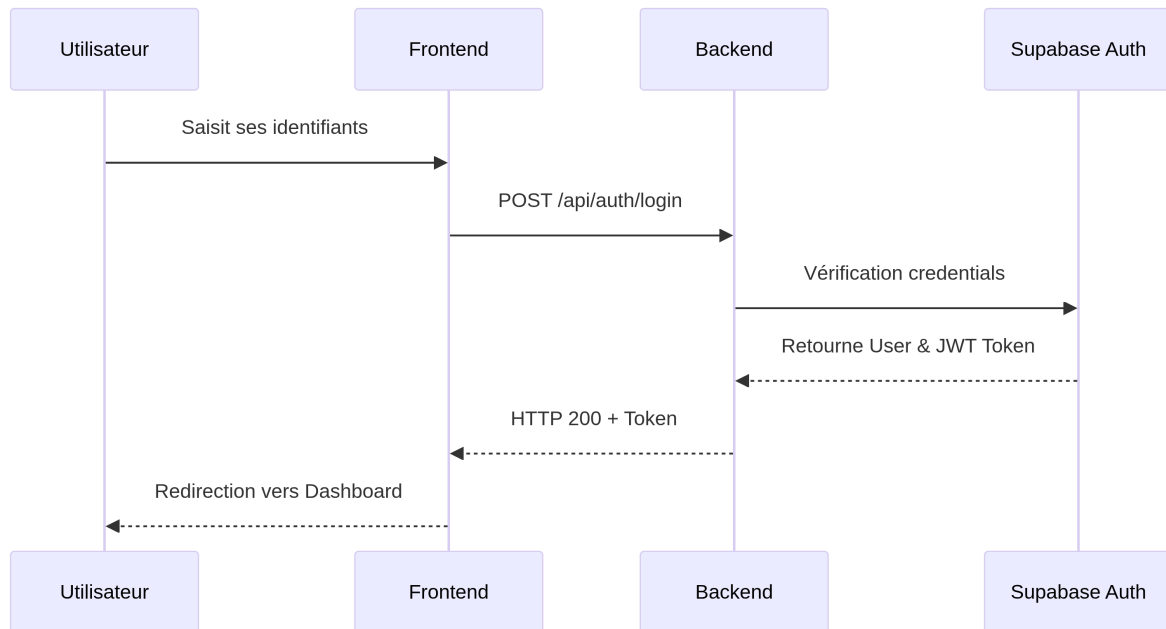
---

## 5. Diagrammes de Séquence (Flux Principaux)

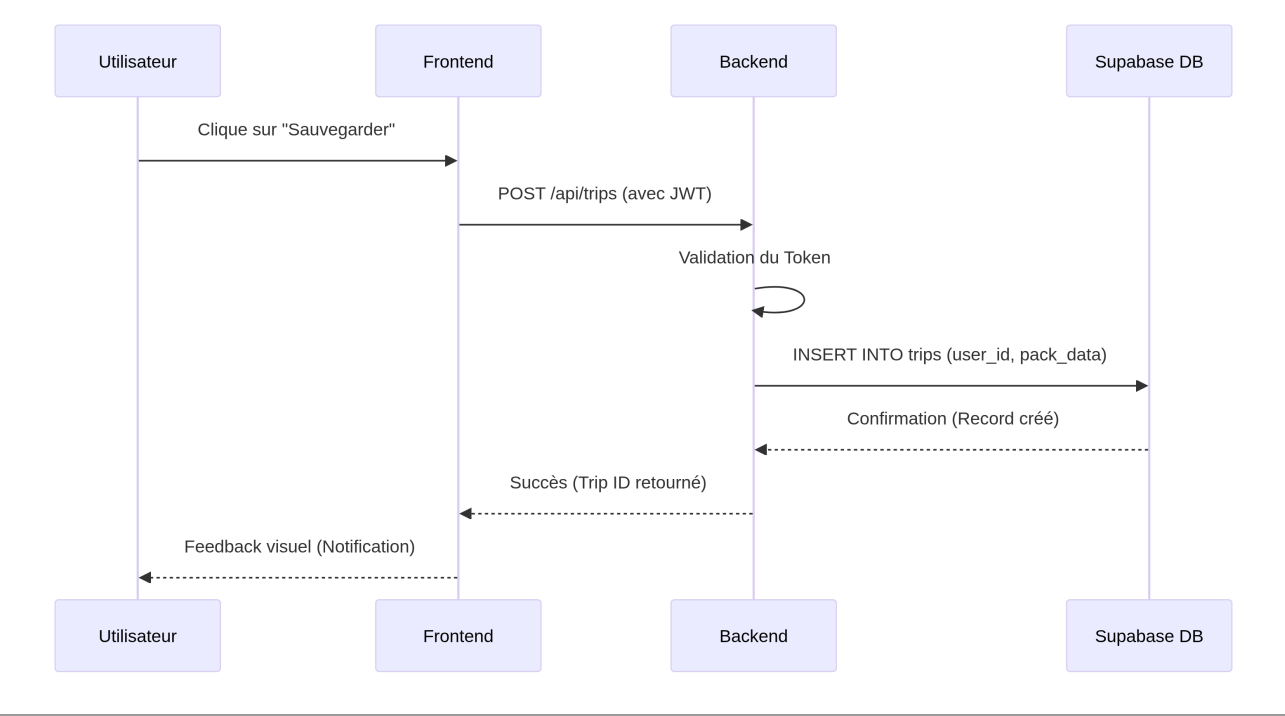
### Cas d'Usage 1 : Génération d'un Itinéraire Signature (IA Agentique)



### Cas d'Usage 2 : Authentification Utilisateur (JWT & Supabase)



### Cas d’Usage 3 : Sauvegarde d’un Voyage (Persistance)



## 6. Spécifications API (Externes & Internes)

### API Externes & Justifications

| Service           | Rôle             | Justification du choix                                                                                         |
|-------------------|------------------|----------------------------------------------------------------------------------------------------------------|
| Claude 3.5 Sonnet | LLM Orchestrator | Supériorité dans le respect des schémas JSON stricts et finesse de la conciergerie VIP.                        |
| Tavily AI         | Web Search Agent | Optimisé pour l’IA, il retourne des données structurées (Vols, Hôtels) sans le “bruit” des moteurs classiques. |
| Unsplash API      | Visual Assets    | Accès à une bibliothèque de photos haute résolution libre de droits via recherche sémantique.                  |
| Supabase          | BaaS (DB & Auth) | Infrastructure PostgreSQL robuste et système d’authentification prêt à l’emploi (Sécurité).                    |

# Endpoints de l'API Interne (TripGenie API)

## Conventions Générales

- **Base URL** : `http://localhost:3000/api`
- **Format** : JSON uniquement ( `Content-Type: application/json` )
- **Auth** : `Authorization: Bearer <JWT_TOKEN>` sur toutes les routes protégées (🔒)
- **Versioning** : `/api/v1/` en production

## 1. Authentification

| Auth   | Méthode | Route                       | Body / Params                    | Réponse                                                             |
|--------|---------|-----------------------------|----------------------------------|---------------------------------------------------------------------|
| Public | POST    | <code>/auth/register</code> | <code>{ email, password }</code> | <code>201 { user_id, email }   409 email déjà utilisé</code>        |
| Public | POST    | <code>/auth/login</code>    | <code>{ email, password }</code> | <code>200 { token, expires_in }   401 identifiants invalides</code> |

## 2. Intelligence Artificielle & Génération

| Auth                                                                                       | Méthode | Route                              | Body / Params                            | Réponse                                                                |
|--------------------------------------------------------------------------------------------|---------|------------------------------------|------------------------------------------|------------------------------------------------------------------------|
| <br>JWT | POST    | <code>/chat/message</code>         | <code>{ session_id?, message }</code>    | <code>200 { reply, session_id, message extracted?, complete }</code>   |
| <br>JWT | GET     | <code>/chat/:session_id</code>     | —                                        | <code>200 { messages[], extracted }</code>                             |
| <br>JWT | POST    | <code>/destinations/suggest</code> | <code>{ session_id }</code>              | <code>200 { destinations: [{name, score, description}] }</code>        |
| <br>JWT | POST    | <code>/packs/generate</code>       | <code>{ session_id, destination }</code> | <code>200 { pack: {flights, hotel, activities, events}, score }</code> |



### 3. Gestion des Voyages (Persistance)

| Auth                                                                                     | Méthode | Route      | Body / Params                | Réponse                        |
|------------------------------------------------------------------------------------------|---------|------------|------------------------------|--------------------------------|
| <br>JWT | POST    | /trips     | { pack, destination, score } | 201 { trip_id, share_token }   |
| <br>JWT | GET     | /trips     | —                            | 200 { trips: [] }              |
| <br>JWT | GET     | /trips/:id | —                            | 200 { trip }   404 not found   |
| <br>JWT | DELETE  | /trips/:id | —                            | 204 No Content   403 Forbidden |

### 4. Vote Collectif

| Auth                                                                                       | Méthode | Route              | Body / Params                                    | Réponse                                   |
|--------------------------------------------------------------------------------------------|---------|--------------------|--------------------------------------------------|-------------------------------------------|
| Public                                                                                     | GET     | /share/:token      | —                                                | 200 { trip }   404   410 lien expiré      |
| Public                                                                                     | POST    | /share/:token/vote | { element_type, element_id, value, voter_token } | 201 { vote_id }   409 déjà voté           |
| <br>JWT | GET     | /trips/:id/results | —                                                | 200 { results: {element_id, up, down}[] } |

## Codes HTTP Standardisés

| Code                             | Signification et usage dans TripGenie                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------------|
| <b>200 OK</b>                    | Requête réussie — retourne les données demandées.                                               |
| <b>201 Created</b>               | Ressource créée avec succès (inscription, sauvegarde voyage, vote).                             |
| <b>204 No Content</b>            | Suppression réussie — aucun corps de réponse.                                                   |
| <b>400 Bad Request</b>           | Données invalides (format email incorrect, budget négatif, champ manquant).                     |
| <b>401 Unauthorized</b>          | Token JWT absent, expiré ou invalide.                                                           |
| <b>403 Forbidden</b>             | Utilisateur authentifié mais sans droit sur la ressource (ex : supprimer le voyage d'un autre). |
| <b>404 Not Found</b>             | Ressource inexistante (voyage introuvable, token de partage inconnu).                           |
| <b>409 Conflict</b>              | Doublon détecté (email déjà utilisé, vote déjà soumis).                                         |
| <b>410 Gone</b>                  | Lien de partage expiré (passé les 7 jours).                                                     |
| <b>429 Too Many Requests</b>     | Rate limiting déclenché (protection anti-abus sur les endpoints IA).                            |
| <b>500 Internal Server Error</b> | Erreur serveur — message générique sans détail sensible exposé au client.                       |

---

## 7. Flux Utilisateur & Wireframes Textuels

### Flux Principal — Happy Path

| Écran                      | Action utilisateur / Comportement système                                                                           |
|----------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1 Landing Page             | L'utilisateur arrive sur TripGenie. CTA "Planifier mon voyage" visible. Bouton "Connexion" en haut à droite.        |
| 2 Inscription / Connexion  | Formulaire email + mot de passe. Validation temps réel. Redirection vers le chatbot après connexion.                |
| 3 Chatbot d'onboarding     | Le chatbot pose 5 questions max. L'utilisateur répond en langage naturel. Indicateur de progression visible.        |
| 4 Suggestions destinations | 3 destinations affichées en cartes avec nom, score % et description courte. L'utilisateur clique sur sa préférée.   |
| 5 Génération du pack       | Loader animé "TripGenie prépare votre voyage...". Pack affiché en moins de 15s : vol, hôtel, activités, événements. |
| 6 Détail du pack           | Vue complète du pack avec score détaillé. Boutons : "Sauvegarder", "Partager au groupe", "Régénérer".               |
| 7 Vote de groupe           | Lien unique partagé. Les amis voient le pack, votent sur chaque élément (👍/👎) sans compte requis.                   |
| 8 Résultats des votes      | L'organisateur voit les votes en temps réel. Il peut valider le pack final et le sauvegarder.                       |
| 9 Mes voyages              | Historique des voyages sauvegardés. Chaque voyage est cliquable pour revoir le pack complet.                        |

### Maquettes UI — Interface Réelle

**Écran 1 : Page d'accueil (Landing Page)** L'utilisateur arrive sur une interface moderne et épurée invitant à la planification de voyages personnalisés.

✦ Voyages de luxe personnalisés

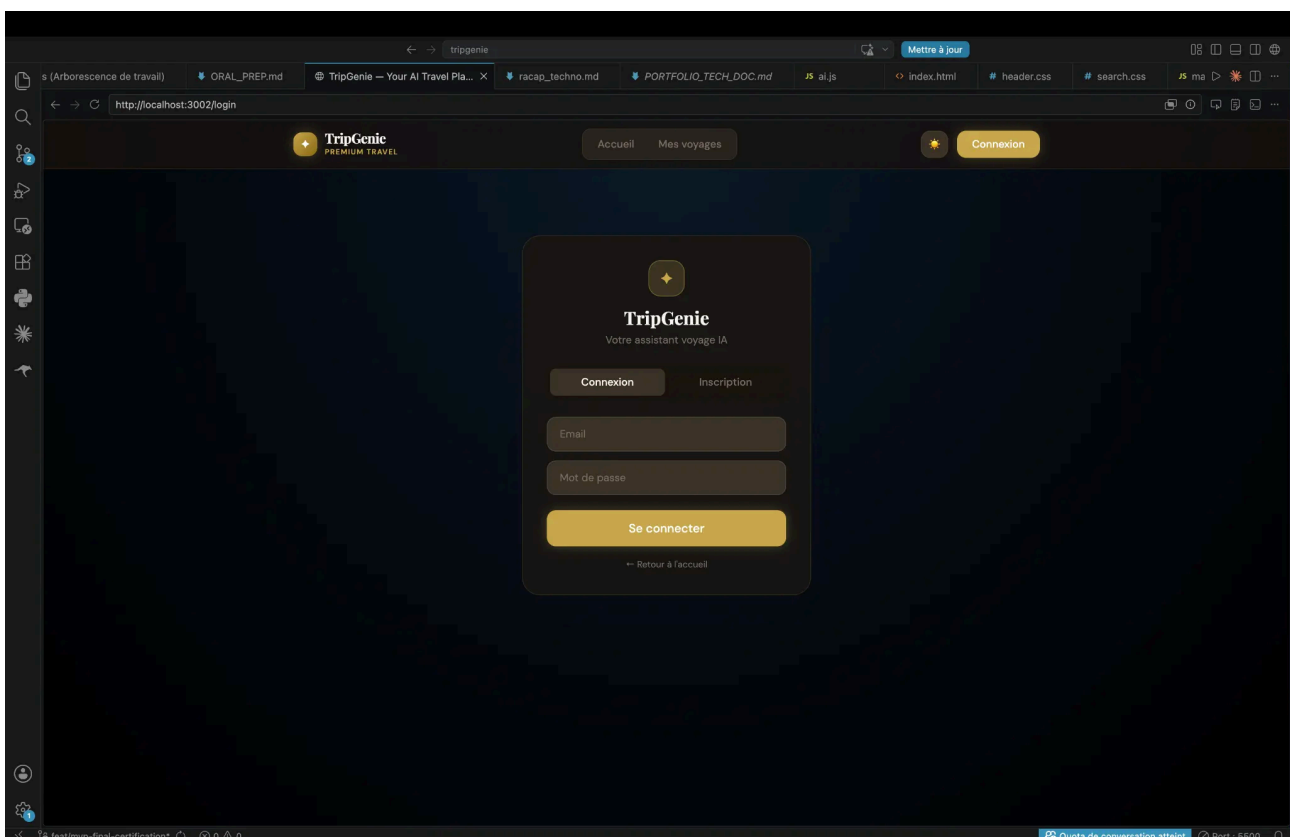
# Créez des voyages inoubliables

TripGenie transforme vos rêves de voyage en réalité. Planifiez, organisez et sharez des expériences uniques avec notre plateforme premium.

Commencer gratuitement

 Explorer

**Écran 2 : Authentification (Connexion)** Interface sécurisée permettant à l'utilisateur d'accéder à son espace personnel et à son historique de voyages.



## 8. Plans SCM & Assurance Qualité (QA)

---

### Gestion du Code Source (SCM)

Le projet utilise **Git** pour le contrôle de version, avec une stratégie de branchement rigoureuse :

- **Branches principales :**
  - `main` : Code stable prêt pour la production.
  - `develop` : Branche d'intégration pour les tests de pré-production.
- **Branches de fonctionnalités :**
  - `feat/*` : Chaque nouvelle tâche est développée sur une branche dédiée (ex: `feat/ai-orchestrator` ) avant d'être fusionnée via une Pull Request.
- **Code Review** : Toutes les demandes d'extraction vers la branche `develop` font l'objet d'une revue systématique.

### Stratégie d'Assurance Qualité (QA)

La fiabilité du système repose sur une approche de tests multi-niveaux :

| Type               | Outil                 | Ce qui est testé                                                | Priorité |
|--------------------|-----------------------|-----------------------------------------------------------------|----------|
| Tests unitaires    | Jest                  | ScoringService, extraction paramètres chatbot, validations Zod  | MUST     |
| Tests intégration  | Jest + Supertest      | Routes HTTP : auth, pack generation, vote — codes retour HTTP   | MUST     |
| Tests E2E (légers) | Manuel / Postman      | Flux complet : inscription → chatbot → pack → sauvegarde → vote | SHOULD   |
| Tests de charge    | Artillery (optionnel) | Résistance API sous 50 utilisateurs simultanés                  | COULD    |
| Tests sécurité     | Manuel                | JWT expiré, injection SQL via Prisma, rate limiting, CORS       | MUST     |

## Exemples de Cas de Tests Critiques

| ID   | Scénario de test                                              | Résultat attendu                  | US liée |
|------|---------------------------------------------------------------|-----------------------------------|---------|
| T-01 | Inscription avec un email déjà utilisé                        | 409 Conflict + message explicite  | US-01   |
| T-02 | Connexion avec mauvais mot de passe                           | 401 sans exposer le hash          | US-02   |
| T-03 | Accès route protégée sans JWT                                 | 401 Unauthorized                  | US-02   |
| T-04 | Chatbot extrait correctement budget=500, group=4, duration=3j | extracted.budget === 500          | US-03   |
| T-05 | Pack généré en moins de 15 secondes                           | response time < 15000ms           | US-05   |
| T-06 | Score calculé pour mode “groupe” avec critères pondérés       | score entre 0 et 100              | US-06   |
| T-07 | Vote avec lien de partage expiré                              | 410 Gone                          | US-08   |
| T-08 | Double vote du même votant sur le même élément                | 409 Conflict                      | US-08   |
| T-09 | Suppression du voyage d’un autre utilisateur                  | 403 Forbidden                     | US-07   |
| T-10 | API LLM indisponible — fallback déclenché                     | Réponse générique sans erreur 500 | US-03   |

## Pipeline de Déploiement (CI/CD)

1. **Local** : Développement et tests unitaires.
  2. **Staging** : Déploiement automatique de `deve1op` sur un environnement de test.
  3. **Production** : Déploiement de `main` après validation finale.
-

## 9. Justifications Techniques & Décisions d'Architecture

---

- **Tavily AI (Recherche Agentique)** : Choisi pour résoudre le problème majeur des LLM : l'obsolescence des données. En couplant Claude 3.5 à Tavily, le système accède à des prix et disponibilités en temps réel, garantissant la fiabilité du MVP.
  - **Claude 3.5 (Orchestration)** : Ce modèle a été retenu pour sa supériorité dans le "Reasoning" et sa capacité à produire du code JSON structuré sans erreurs, indispensable pour la stabilité du Frontend.
  - **Supabase (Infrastructure PostgreSQL)** :
    - **Choix stratégique** : Plutôt que de consacrer du temps à la maintenance d'un serveur DB Linux, le choix s'est porté sur une solution managée pour garantir une sécurité maximale (Authentification JWT, Chiffrement des données au repos).
    - **Respect des fondamentaux** : Bien que le service soit managé, la **conception relationnelle** (MCD/MLD) a été entièrement réalisée à la main (Schéma ERD). L'utilisation de PostgreSQL standard garantit la portabilité totale du projet vers n'importe quel autre hébergeur SQL si nécessaire.
    - **Sécurité** : L'utilisation de Supabase Auth permet de respecter les standards de sécurité actuels (OAuth2, Sessions sécurisées), une priorité absolue pour une application gérant des itinéraires personnels.
-

# 10. Résumé & Prochaines Étapes

## Livrables de la Phase 3

| Livrable               | Contenu                                                                | Statut    |
|------------------------|------------------------------------------------------------------------|-----------|
| User Personas          | 3 personas détaillés avec comportements et frustrations                | ✓ Terminé |
| User Stories           | 8 US priorisées MoSCoW avec critères d'acceptation et endpoints        | ✓ Terminé |
| Architecture système   | Schéma 3 couches + stack technique justifiée (12 technologies)         | ✓ Terminé |
| Schéma base de données | 4 tables PostgreSQL avec colonnes, types, contraintes et relations ERD | ✓ Terminé |
| Specs API REST         | 15 endpoints documentés avec méthodes, body, réponses et codes HTTP    | ✓ Terminé |
| Wireframes             | Flux utilisateur complet (9 étapes) + 2 wireframes textuels détaillés  | ✓ Terminé |
| Stratégie de tests     | 5 types de tests + 10 cas de tests critiques avec résultats attendus   | ✓ Terminé |

## Prochaines Étapes — Phase 4 (Semaines 7–10)

- Mise en place de l’environnement de développement (repo GitHub, CI/CD Railway, variables d’environnement).
- Initialisation du projet : React + Vite (frontend), Node.js + Express + Prisma (backend).
- Développement Sprint 1 (Sem. 7) : Authentification JWT complète + structure BDD PostgreSQL.
- Développement Sprint 2 (Sem. 8) : Chatbot IA (intégration LLM) + endpoint suggestions destinations.
- Développement Sprint 3 (Sem. 9) : Génération pack voyage + scoring + interface React connectée.



- Développement Sprint 4 (Sem. 10) : Vote collectif + tests automatisés + déploiement production.